

RegiMon: Real-Time Monitoring of the Volatile Windows Registry

1st Anik Bakshi
Academies of Loudoun
Information Technology
Leesburg, Virginia
anik.x.bakshi@gmail.com

2nd Arush Bodla
Academies of Loudoun
Information Technology
Leesburg, Virginia
arushbodla@gmail.com

Abstract

The Windows Registry is a hierarchical database containing key-value pairs governing core operating system behavior including security configurations, user permissions, and software privileges. Because of its central role in system operation, the Registry is a prime target for malware seeking to establish persistence. The Windows Registry is stored on disk, but when the system boots, a volatile copy of the registry is created in RAM which then becomes the working copy. Existing tools such as RegRipper or RegShot analyze the on-disk registry through point-in-time snapshots; however, there is a significant gap in monitoring the volatile registry which allows malicious activity to go unnoticed.

This project presents RegiMon, a lightweight real-time registry monitoring application built in Rust using the Tauri framework. RegiMon takes advantage of the Windows API call `RegNotifyChangeKeyValue()` to continuously monitor the volatile RAM copy of the registry, detecting modifications, deletions, and insertions to the registry as they occur. Upon detecting a change, the application alerts the user and offers immediate remediation options, including the ability to undo suspicious modifications. RegiMon runs as a system tray application, minimizing performance overhead and avoiding interference with existing security software.

Testing was conducted in a sandboxed Windows 10 environment using manual registry edits and system settings changes. RegiMon successfully detected changes across 25 keys in real time, demonstrating its viability as an early detection mechanism. Following these tests, the ILOVEYOU virus was used against RegiMon, and RegiMon was able to prevent the persistence mechanisms ILOVEYOU attempted to create. Future work will extend monitoring to a larger set of keys and implement persistent logging.

Index Terms

Windows Registry, malware detection, real-time monitoring, Rust, endpoint security, `RegNotifyChangeValue`, `Win32`

I. INTRODUCTION

The Windows operating system has a registry that contains key values which control certain settings about the computer (1). Normal software uses this to determine things such as permissions and policy enforcement, but malware also has the ability to modify the registry. Because of how large the registry system is, it's very difficult to determine if there are security vulnerabilities in the machine and prevent them.

Additionally, there are two copies of the Windows Registry: one that exists on-disk and one that exists in volatile memory (RAM). The on-disk copy is the storage and only changes on the restart of a computer and the in-memory copy is the working version used by the kernel and software (2).

Currently, software that is built around monitoring the registry typically use the on-disk copy to monitor changes (3). This leaves the memory copy to be unmonitored and malicious programs can cause damage to the computer (4). Current software can also show registry edits or file changes, but don't adequately group them into a level of threat or potential issues and solutions (5).

For example, RegRipper is a popular tool used to export hives as DAT files and read into each hive. However, hives contain thousands of keys and the user can't look for anything specifically unless they manually go in and find the specific keys they wanted to check out of the entire hive. This makes the process inefficient and more difficult to spot malicious intent.

Another popular tool is called RegShot. RegShot works by storing a state of the entire windows registry as one snapshot then allowing the user to take another snapshot and compare what has changed. The downside to this solution is that the registry consists of a few million keys and takes a while to save one snapshot. By that time, malicious programs can change parts of the windows registry and compromise the computer. Additionally, looking at every single key is not efficient because just by simply using Windows, thousands of keys change per day and as a result there will be many changes in the comparison of the 2 snapshots.

Another program that exists is called Volatility. Unlike RegRipper and RegShot which use the disk, volatility reads into the RAM by storing a copy of RAM dumps to be analyzed by the user. However, these RAM dumps are only available after dumping the RAM which means it cannot be monitored in real-time. Again, this allows malicious code to damage the computer.

This paper outlines the development and use for Regimon which is able to cover the significant gaps in current softwares by monitoring the Registry in real-time, using the memory copy, allowing specific keys to be picked for monitoring, and being able to undo any changes done by malicious code.

The use of a virus, specifically the ILOVEYOU virus, was used in a virtual machine to test RegiMon. The ILOVEYOU virus was from the early 2000s that targeted the email address of a windows machine. Upon opening the .vbs file attached in an email, the virus would spread itself to other machines by sending emails of itself from the user's account and changed registry keys in order to achieve this goal (6). This virus will be used for early testing of Regimon due to its low danger in Windows 10 systems and the scale at which it compromised machines at it's peak.

II. METHODOLOGY

A. Development Environment Setup

Development required installing the Rust toolchain using the Rustup installer. Then, the Node Package Manager (npm) was used to install the Tauri and React frameworks. The npm setup included Tauri 2.8.0, Tailwind CSS 4.1.17, Vite 7.3.1, and React 19.1.1. Cargo managed Rust dependencies and crates including `serde` for serialization of registry changes, `winreg` to open and interact with Registry keys, and `windows-sys` to access the Win32 API and invoke `RegNotifyChangeKeyValue()`. Visual Studio Code was the primary development environment, with the `tauri dev` and `tauri build` commands used for development (with hot reload) and production builds respectively. CI/CD was setup with a GitHub Actions workflow so that any commit with a tag starting with `v` was automatically built using Tauri and pushed as a GitHub Release. The code and releases were published at <https://github.com/ArushNo1/RegiMon>.

B. System Architecture

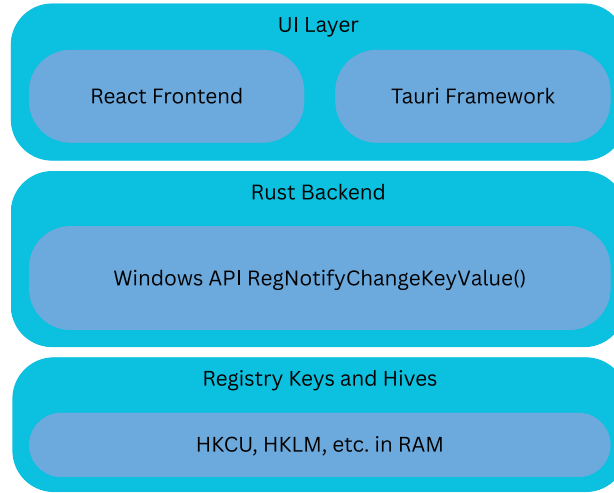


Fig. 1. Diagram of System Architecture of RegiMon.

RegiMon is organized into three layers: the Windows Registry, a Rust backend, and a React frontend connected via Tauri. The backend spawns a dedicated thread per monitored key, each blocking on a `RegNotifyChangeKeyValue()` call. When the Windows API wakes the thread upon detecting a registry change, the backend reads and categorizes the modification, then emits a Tauri event to the frontend. The frontend renders the change in the Recent Changes panel in the user interface and exposes an undo action, which invokes the `undo_registry_change` Tauri command, triggering a write-back on the backend.

C. Testing and Validation

Testing was conducted by running RegiMon alongside registry modification utilities. The testing environment was a Windows 10 virtual machine hosted on an Ubuntu 22.04 ThinkStation, set up using VMware Workstation Pro 17. The virtual machine had 2 GB of memory, 2 processor cores, and 60 GB of disk storage. The virtual machine was given internet access through an ethernet connection on the host machine, which was then disconnected after installation completed to isolate any unintended effects of running Love Bug.

First, the time zone of the computer was changed using Windows Settings, causing modifications to the `TimeZoneInformation` key.¹ Next, manual registry edits were introduced using the built-in Windows `regedit` utility to test detection of direct key modifications.

¹`HKLM\SYSTEM\CurrentControlSet\Control\TimeZoneInformation`

The final stage of testing was conducted with ILOVEYOU. The source code for the worm was pulled from <https://github.com/onx/ILOVEYOU>. The if-statements in the vbs script were then compressed from multiple lines to one line to comply with the Windows 10 syntax. The `Start Page` and `Timeout` keys (see Table II) were modified to the updated Windows 10 paths. Windows defender was turned off in order to let RegiMon and ILOVEYOU execute. A VMWare snapshot of the Windows 10 machine was created. Finally, ILOVEYOU was executed through an elevated command prompt in order to validate.

RegiMon has reached a functional state as a real-time Windows Registry monitoring application. The core monitoring engine, built in Rust using the `winreg` and `windows-sys` crates, leverages `RegNotifyChangeKeyValue()` to detect changes as they occur rather than relying on periodic snapshots. The application tracks modifications, additions, and deletions across a configurable set of registry keys and hives.

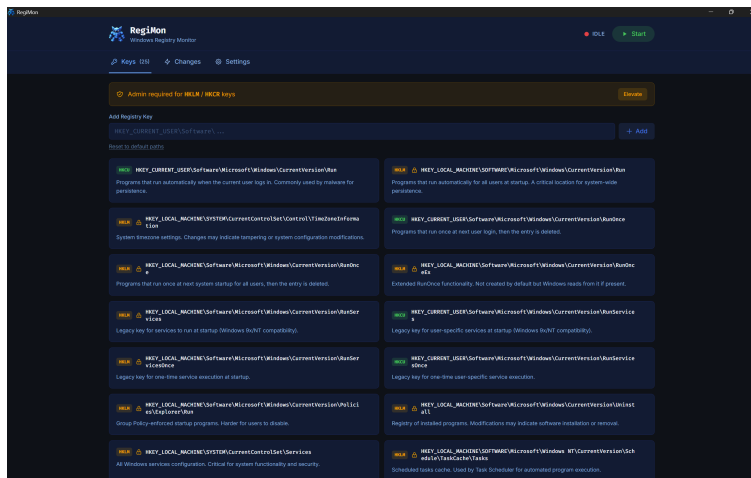


Fig. 2. RegiMon interface showing the list of currently monitored keys

Of the 25 registry keys selected for monitoring, 17 were confirmed to exist on at least one test machine. The remaining 8 keys were legacy or optional paths not present on Windows 10 by default, including compatibility keys from the Windows 9x and NT era such as `RunServices` and `RunServicesOnce`. Of the 17 existing keys, 11 were validated on the development machine and 12 on the Windows 10 virtual machine, with 9 keys validated on both. RegiMon successfully detected changes on all keys that existed and were tested, yielding a detection rate of 100% across verified test cases. A representative sample is shown in Table I and the complete key set with per-machine coverage is provided in Appendix A (Table III).

Registry Key	Purpose	Dev	VM
... \Run	Startup persistence	✓	✓
... \RunOnce	One-time startup execution	✓	✓
... \TimeZoneInformation	System time configuration	×	✓
... \RunServicesEx	Legacy service persistence	×	×
... \Winlogon	Login behavior configuration	✓	×

C. Testing Results

Testing was conducted across three scenarios: manual registry edits via regedit, a system-level settings change, and execution of the ILOVEYOU virus. In all tested scenarios, RegiMon successfully detected targeted registry changes and surfaced them in the Recent Changes tab (Figure 4). The monitor identified the changed key path, the type of change, and provided context about the key’s sensitivity. Undo functionality was verified for all flagged changes where applicable.

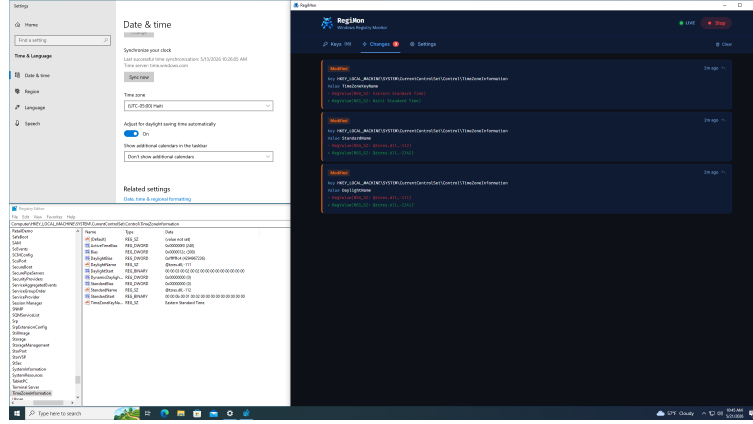


Fig. 3. RegiMon interface showing the Recent Changes tab active, with the Windows Settings Date & Time pane and Registry Editor open to TimeZoneInformation, demonstrating real-time detection of a system-level registry change.

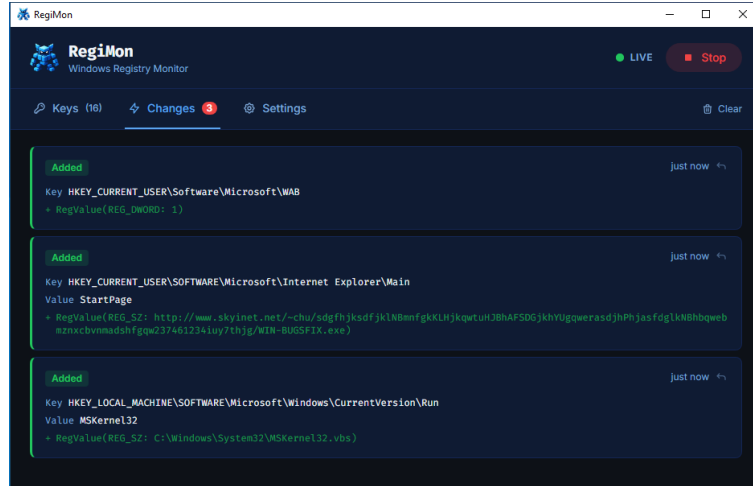


Fig. 4. ILOVEYOU monitored key changes

D. ILOVEYOU Virus Test

RegiMon successfully detected and remediated the registry modifications attempted by the ILOVEYOU virus. Table II details each registry modification ILOVEYOU attempted, whether it constituted an active change, and whether the undo operation succeeded. The only artifact remaining after remediation was a copy of the virus binary located at `C:\Windows\System32\MSKernel32.vbs`, which exists outside the registry and therefore outside RegiMon’s current scope.

IV. DISCUSSION

A. Design Goal Assessment

The primary design goal of RegiMon was to create a continuously running, real-time registry monitor capable of detecting changes and alerting users so they could take corrective action. This goal was substantially met. The application monitors a configurable set of registry keys in real time, detects modifications, additions, and deletions, and provides an undo mechanism. The production build runs as a system tray application with an installer, making it suitable for deployment on a standard Windows machine. A known limitation is the incomplete notification system.

TABLE II
REGISTRY MODIFICATIONS IN ILOVEYOU

Registry Key::Value	Modification	Undo Successful
HKCU\Software\Microsoft\Windows Scripting Host\Settings::Timeout	None	N/A
HKLM\Software\Microsoft\Windows\CurrentVersion\Run	Created MSKernel32.vbs	Yes
HKLM\Software\Microsoft\Windows\CurrentVersion\RunServices	None	N/A
HKCU\Software\Microsoft\Internet Explorer::Download Directory	None	N/A
HKCU\Software\Microsoft\Internet Explorer\Main::Start Page	Created malicious link	Yes
HKCU\Software\Microsoft\WAB::(default)	Updated 0 → 1	Yes

B. Improvement Over Existing Solutions

Tools such as RegRipper, RegShot, and Volatility share a common limitation: they are point-in-time tools requiring manual invocation with no continuous monitoring capability. RegRipper extracts forensic details from hive files; RegShot compares two snapshots taken at different times; Volatility analyzes memory dumps taken at a single point in time, meaning it can inspect the volatile registry but only after the fact. None provide real-time, event-driven detection. By using `RegNotifyChangeKeyValue()`, RegiMon is notified of changes as they occur rather than requiring a snapshot or dump, making it better suited for proactive threat detection than post-incident forensic analysis. Additionally, traditional antivirus software operates opaquely, concealing which keys or behavior patterns it monitors, whereas RegiMon provides full visibility into what is being watched and why, making it particularly useful for security research.

C. Applications and Recommendations

RegiMon is suited for endpoint security monitoring, security research sandboxes, and individual users concerned about unauthorized changes to startup or configuration keys. The recommended use case for the current build is controlled environments, such as security labs, classroom research, and virtualized sandboxes, where monitored keys can be carefully configured and false positives manually reviewed. RegiMon is recommended as a first-response tool in sandbox malware analysis environments.

D. New Research Questions

Several questions are worth further investigation: How necessary is monitoring legacy keys that no longer exist on modern versions of Windows? Can machine learning classify registry change events as malicious or benign more accurately than rule-based matching? How would RegiMon's performance compare against malware that doesn't rely as heavily on the registry as ILOVEYOU?

E. Future Testing and Development

Additional testing and development are required before RegiMon is suitable for deployment outside of controlled environments. The current test suite covered manual registry edits, system-level changes, and a single malware sample. While these results are promising, a broader set of malware samples representing modern persistence techniques is needed to assess generalizability. The monitored key set should also be expanded, as malware targeting keys outside the current set would evade detection entirely. On the development side, the notification system requires stabilization and persistent logging must be implemented. Until these are addressed, RegiMon is best treated as a research prototype rather than a production security tool.

F. Next Steps

Several directions remain for future development. The notification system requires stabilization so that alerts are delivered passively without requiring manual interaction with the interface. Monitoring coverage should be extended to a broader set of registry hives to reduce the likelihood of evasion by malware targeting keys outside the current monitored set. A structured persistent logging system would allow RegiMon to record all change events to disk, enabling post-incident review. Finally, additional malware samples beyond the Love Bug should be tested to assess how well RegiMon generalizes to modern persistence techniques.

ACKNOWLEDGMENT

The authors thank Matthew Bailey of Brixen Development and Matt Quinn of Data Machines for recognition of this work at the 44th Loudoun County Regional Science and Engineering Fair.

The authors thank Dr. Gerald Shapiro and Dr. Patrick White of the Academies of Loudoun for mentoring them throughout the process of this research project.

REFERENCES

- [1] Microsoft, “Windows registry for advanced users,” *Microsoft Learn*. [Online]. Available: <https://learn.microsoft.com/en-us/troubleshoot/windows-server/performance/windows-registry-advanced-users>. [Accessed: Sep. 2024].
- [2] B. Dolan-Gavitt, “Forensic analysis of the Windows registry in memory,” *Digital Investigation*, vol. 5, pp. S26–S32, 2008. doi: 10.1016/j.diin.2008.05.003
- [3] MITRE ATT&CK, “Boot or logon autostart execution: Registry run keys / startup folder,” ATT&CK Technique T1547.001. [Online]. Available: <https://attack.mitre.org/techniques/T1547/001>. [Accessed: Oct. 2024].
- [4] Picus Security, “MITRE ATT&CK T1060: Registry run keys / startup folder,” *Picus Security Blog*. [Online]. Available: <https://www.picussecurity.com/resource/blog/picus-10-critical-mitre-attck-techniques-t1060-registry-run-keys-startup-folder>. [Accessed: Oct. 2024].
- [5] Red Canary, “Windows registry attacks: Threat detection,” *Red Canary Blog*. [Online]. Available: <https://redcanary.com/blog/threat-detection/windows-registry-attacks-threat-detection>. [Accessed: Oct. 2024].
- [6] Splunk, “From registry with love: Malware registry abuses,” *Splunk Security Blog*. [Online]. Available: https://www.splunk.com/en_us/blog/security/from-registry-with-love-malware-registry-abuses.html. [Accessed: Oct. 2024].

APPENDIX A: COMPLETE MONITORED REGISTRY KEY SET

TABLE III
COMPLETE MONITORED REGISTRY KEYS AND TEST COVERAGE

Registry Key	Purpose	Dev	VM
HKCU\Software\Microsoft\Windows\CurrentVersion\Run	Programs that run automatically when the current user logs in. Commonly used by malware for persistence.	✓	✓
HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\Run	Programs that run automatically for all users at startup. A critical location for system-wide persistence.	✓	✓
HKLM\SYSTEM\CurrentControlSet\Control\TimeZoneInformation	System timezone settings. Changes may indicate tampering or system configuration modifications.	x	✓
HKCU\Software\Microsoft\Windows\CurrentVersion\RunOnce	Programs that run once at next user login, then the entry is deleted.	✓	x
HKLM\Software\Microsoft\Windows\CurrentVersion\RunOnce	Programs that run once at next system startup for all users, then the entry is deleted.	✓	x
HKLM\Software\Microsoft\Windows\CurrentVersion\RunOnceEx	Extended RunOnce functionality. Not created by default but Windows reads from it if present.	x	x
HKLM\Software\Microsoft\Windows\CurrentVersion\RunServices	Legacy key for services to run at startup (Windows 9x/NT compatibility).	x	✓
HKCU\Software\Microsoft\Windows\CurrentVersion\RunServices	Legacy key for user-specific services at startup (Windows 9x/NT compatibility).	x	✓
HKLM\Software\Microsoft\Windows\CurrentVersion\RunServicesOnce	Legacy key for one-time service execution at startup.	x	x
HKCU\Software\Microsoft\Windows\CurrentVersion\RunServicesOnce	Legacy key for one-time user-specific service execution.	x	x
HKLM\Software\Microsoft\Windows\CurrentVersion\Policies\Explorer\Run	Group Policy-enforced startup programs. Harder for users to disable.	x	x
HKLM\Software\Microsoft\Windows\CurrentVersion\Uninstall	Registry of installed programs. Modifications may indicate software installation or removal.	✓	x
HKLM\SYSTEM\CurrentControlSet\Services	All Windows services configuration. Critical for system functionality and security.	✓	x
HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Schedule\TaskCache\Tasks	Scheduled tasks cache. Used by Task Scheduler for automated program execution.	x	x
HKLM\Software\Microsoft\Windows NT\CurrentVersion\Winlogon	Login process configuration. Often targeted by malware to control user sessions.	x	x
HKCU\Software\Microsoft\Windows\CurrentVersion\Internet Settings	Internet Explorer settings for current user including proxy configuration.	✓	✓
HKLM\Software\Microsoft\Windows\CurrentVersion\Internet Settings	System-wide Internet Explorer settings and proxy configuration.	x	✓
HKLM\Software\Microsoft\WBEM	Windows Management Instrumentation settings. Used for system management and monitoring.	x	✓
HKCU\CLSID	COM class registrations. Modifications can redirect application behavior or enable attacks.	x	x
HKLM\System\CurrentControlSet\Control\Lsa	Local Security Authority configuration. Controls authentication and security policies.	x	x
HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Windows\AppInit_DLLs	DLLs loaded into every process. Heavily abused by malware for code injection.	✓	✓
HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Image File Execution Options	Debugging settings and executable redirections. Can be used to hijack program execution.	✓	x
HKCU\Environment	User-specific environment variables. Changes can affect user sessions and application behavior.	✓	✓
HKCU\Software\Microsoft\WAB	Modified by lovebug worm to keep track of emails sent	✓	✓
HKCU\SOFTWARE\Microsoft\Internet Explorer\Main	Used by lovebug worm to make homepage download another virus	✓	✓